

Cloud Native Architecture

Practical Manual with Solutions and Real Console Screenshots

MCA Sem 1 | PICA | Prof. Shivam Bhakta | Google Cloud Platform

Each practical lives in 5 LAB PRACTICAL MANUALS WITH SOLUTIONS/practicals/practical_XX/ with its .txt manual and images/ folder. Steps without a unique screenshot show [Screenshot pending] and a console deep link.

Index of Practicals

Practical	Title	Steps	Console home
P1	Create The Cloud Account And Explore Cloud Console And Basic Services	25	https://console.cloud.google.com/
P2	Create And Configure Vm Instances On Google Cloud Platform	30	https://console.cloud.google.com/compute/instances
P3	Create Buckets In Cloud Storage	26	https://console.cloud.google.com/storage/browser
P4	Create Users, Roles, And Permissions	28	https://console.cloud.google.com/iam-admin/iam
P5	Create Docker Images And Containers And Run Containerized Applications	35	https://console.cloud.google.com/compute/instances
P6	Create A Kubernetes Cluster And Deploy Applications Using Pods And Services	31	https://console.cloud.google.com/kubernetes/list
P7	Design And Deploy Microservices Using Cloud Run	36	https://console.cloud.google.com/run
P8	Automate Build And Deployment Pipeline	34	https://console.cloud.google.com/cloud-build/builds
P9	Use Cloud	37	https://

	Monitoring And Logging Tools		console.cloud.google.com/logs/query
P10	Develop And Deploy Serverless Functions	28	https://console.cloud.google.com/run

Practical 1: Create The Cloud Account And Explore Cloud Console And Basic Services

Objective: In this practical, you will create or access a cloud account, understand the cloud console interface, identify basic cloud services, and learn safe first steps before creating chargeable resources.

STEP 1: Open Google Cloud website

- What to do:

Open a web browser and go to <https://cloud.google.com>

Solution: Expected result: The Google Cloud home page opens.

Open in Google Cloud Console: <https://cloud.google.com>

On screen: Open the Google Cloud marketing website in a browser

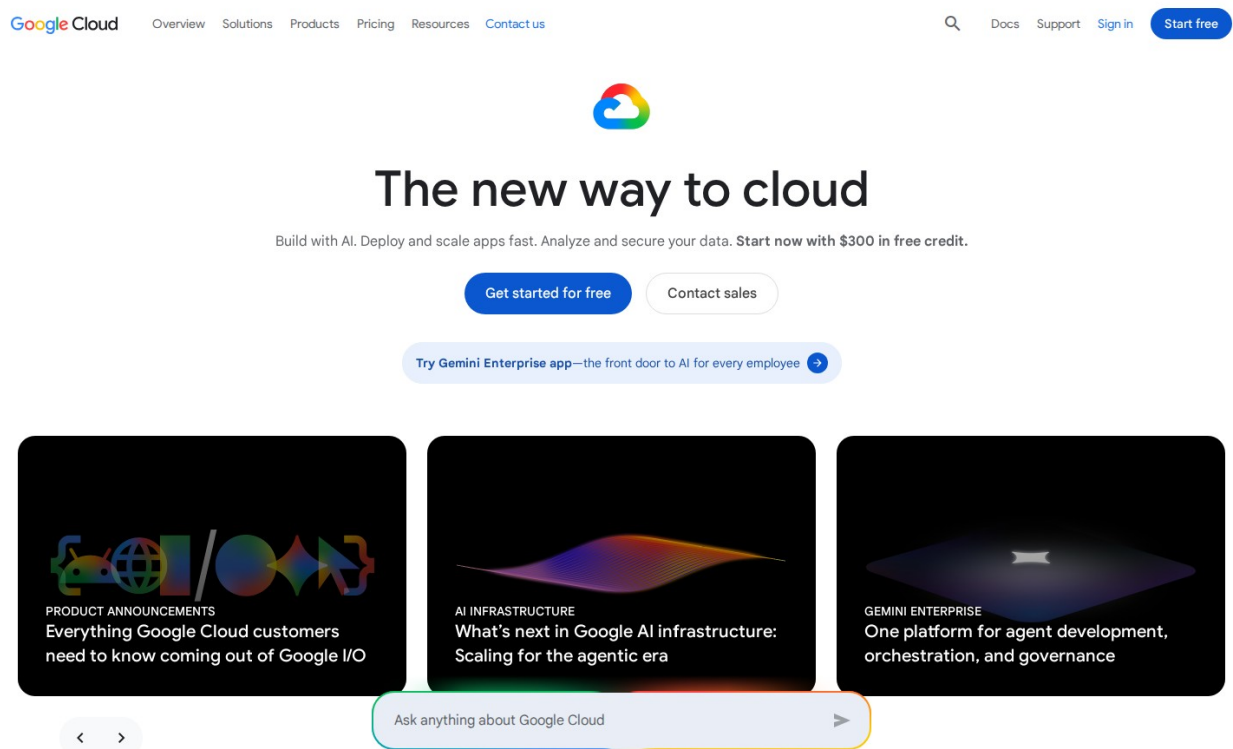


Figure: Real Google Cloud Console screenshot for this step.

STEP 2: Open Google Cloud Console

- What to do:

Click Console or open <https://console.cloud.google.com>

Solution: Expected result: The Google Cloud Console sign-in page or dashboard appears.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Navigate to Google Cloud Console from the website or direct URL

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 3: Sign in with a Google account

- What to do:

Sign in using your Google account or institution-provided account

Solution: Expected result: You are signed in to Google Cloud Console.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Sign in to Google Cloud Console with your Google account

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 4: Accept terms if asked

- What to do:

Read and accept terms of service if prompted

Solution: Expected result: You can continue to the console.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Accept Google Cloud terms of service if prompted

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 5: Check the top navigation bar

- What to do:

Identify search bar, project selector, notifications, Cloud Shell, help, and profile icons

Solution: Expected result: You can identify the main navigation controls.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Identify top navigation bar elements in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 6: Locate the project selector

- What to do:

Find and click the project selector near the top of the console

Solution: Expected result: A project selection window opens.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Click the project selector in the top navigation bar

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 7: View existing projects

- What to do:

Check whether any projects already exist in the project selector

Solution: Expected result: You can see a list of available projects or an option to create a project.

Open in Google Cloud Console: <https://console.cloud.google.com/cloud-resource-manager>

On screen: View existing projects in the project selection dialog

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 8: Create a project only if required

- What to do:

Click New Project, enter name cloud-native-practical, click Create

Solution: Expected result: A new project is created and selected.

Open in Google Cloud Console: <https://console.cloud.google.com/projectcreate>

On screen: Tap New Project button and create project named cloud-native-practical

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 9: Confirm the selected project

- What to do:

Verify the project name shown at the top of the console

Solution: Expected result: The correct project name is visible in the top navigation bar.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Confirm the selected project name in the top navigation bar

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 10: Open the navigation menu

- What to do:

Click the hamburger menu icon in the top-left corner

Solution: Expected result: The main Google Cloud navigation menu opens.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Open the navigation menu using the hamburger icon

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 11: Explore Compute Engine

- What to do:

Search for Compute Engine and open it; do not create a VM

Solution: Expected result: You can view the Compute Engine section. Observation: Compute Engine is used for creating VM instances.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Search and open Compute Engine in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 12: Explore Cloud Storage

- What to do:

Search for Cloud Storage and open it; do not create a bucket

Solution: Expected result: You can view the Cloud Storage section. Observation: Cloud Storage is used for object storage.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Search and open Cloud Storage in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 13: Explore IAM

- What to do:

Search for IAM and open IAM and Admin

Solution: Expected result: You can view IAM and permission-related options.

Observation: IAM is used to manage users, roles, and permissions.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Search and open IAM and Admin in Google Cloud Console

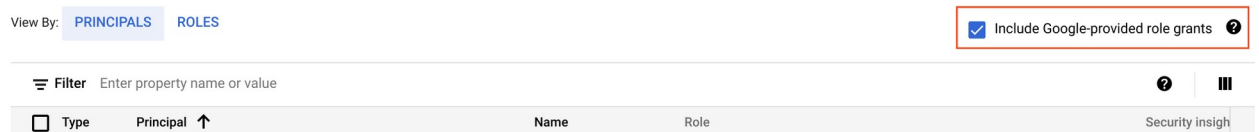


Figure: Official Google Cloud documentation screenshot for this screen. Run `capture_gcp_practical_screenshots.py --login --practical N` for a live capture.

STEP 14: Explore Cloud Run

- What to do:

Search for and open Cloud Run

Solution: Expected result: You can view the Cloud Run page. Observation: Cloud Run can run containerized applications in a managed serverless environment.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Search and open Cloud Run in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 15: Explore Kubernetes Engine

- What to do:

Search for Kubernetes Engine and open it; do not create a cluster

Solution: Expected result: You can view the Kubernetes Engine section.

Observation: GKE is used to create and manage Kubernetes clusters.

Open in Google Cloud Console: <https://console.cloud.google.com/kubernetes/list>

On screen: Search and open Kubernetes Engine in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 16: Explore Cloud Functions

- What to do:

Search for and open Cloud Functions

Solution: Expected result: You can view the Cloud Functions section. Observation: Cloud Functions is used for event-driven serverless functions.

Open in Google Cloud Console: <https://console.cloud.google.com/functions/list>

On screen: Search and open Cloud Functions in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 17: Explore Cloud Monitoring

- What to do:

Search for and open Cloud Monitoring

Solution: Expected result: You can view monitoring dashboards or monitoring setup options. Observation: Cloud Monitoring is used to observe resource health and performance metrics.

Open in Google Cloud Console: <https://console.cloud.google.com/monitoring>

On screen: Search and open Cloud Monitoring in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 18: Explore Cloud Logging

- What to do:

Search for and open Cloud Logging

Solution: Expected result: You can view the logging section. Observation: Cloud Logging is used to view and analyze logs.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Search and open Cloud Logging in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 19: Open Cloud Shell

- What to do:

Click the Cloud Shell icon and wait for it to start

Solution: Expected result: A terminal opens at the bottom or in a new window.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Click the Cloud Shell icon in the top-right of the console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 20: Run a basic Cloud Shell command

- What to do:

In Cloud Shell run: `gcloud --version`

Solution: Expected result: Version information for Google Cloud CLI appears.

Command: `gcloud --version`

STEP 21: Check active account in Cloud Shell

- What to do:

Run: `gcloud auth list`

Solution: Expected result: Your signed-in account appears in the output.

Command: `gcloud auth list`

STEP 22: Check active project in Cloud Shell

- What to do:

Run: `gcloud config get-value project`

Solution: Expected result: The selected project ID appears.

Command: `gcloud config get-value project`

STEP 23: Review billing section carefully

- What to do:

Search for Billing and open it; do not add payment details unless instructed

Solution: Expected result: You can identify whether billing is enabled, disabled, or managed by your organization.

Open in Google Cloud Console: <https://console.cloud.google.com/billing>

On screen: Search and open Billing section in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 24: Review quotas section

- What to do:

Search for Quotas and open the quotas page

Solution: Expected result: You can view quota-related information for cloud services.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/quotas>

On screen: Search and open Quotas page in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

STEP 25: Do not create unnecessary resources

- What to do:

Verify no VM instances, clusters, buckets, Cloud Run services, functions, or databases were created

Solution: Expected result: No unnecessary paid resources are running.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Verify no unintended resources were created in the console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 1` on a machine with your GCP account to capture this step.

Practical 2: Create And Configure Vm Instances On Google Cloud Platform

Objective: In this practical, you will create, configure, connect to, inspect, stop, start, and delete a virtual machine instance on Google Cloud Platform using Compute Engine.

STEP 1: Open Google Cloud Console

- What to do:

Open <https://console.cloud.google.com> and sign in

Solution: Expected result: Google Cloud Console opens successfully.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Open and sign in to Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 2: Select the correct project

- What to do:

Use the project selector to choose the practical project

Solution: Expected result: The correct project name is visible in the top navigation bar.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Select the correct project using the project selector

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 3: Open Compute Engine

- What to do:

Search for Compute Engine and open it

Solution: Expected result: The Compute Engine page opens.

Open in Google Cloud Console: <https://console.cloud.google.com/compute>

On screen: Search and open Compute Engine in the console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 4: Enable Compute Engine API if asked

- What to do:

Click Enable and wait for initialization

Solution: Expected result: Compute Engine becomes available for the project.

Open in Google Cloud Console: <https://console.cloud.google.com/compute>

On screen: Tap Enable button to activate Compute Engine API

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 5: Open VM instances page

- What to do:

In Compute Engine click VM instances

Solution: Expected result: The VM instances list page opens.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Open VM instances page in Compute Engine

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 6: Click Create Instance

- What to do:

Click Create Instance to start the VM creation form

Solution: Expected result: The Create an instance page opens.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Tap Create Instance button in Compute Engine

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 7: Enter VM name

- What to do:

Enter VM name: cloudnative-vm-1

Solution: Expected result: The VM name is entered.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Enter VM name cloudnative-vm-1 in the Create Instance form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 8: Select region and zone

- What to do:

Choose region us-central1 and zone us-central1-a as instructed

Solution: Expected result: Region and zone are selected.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Select region and zone in the VM creation form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 9: Select machine type

- What to do:

Choose E2 series, e2-micro or e2-small

Solution: Expected result: A suitable small machine type is selected.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Select e2-micro or e2-small machine type

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 10: Configure boot disk

- What to do:

Choose Debian Linux, balanced persistent disk, 10 GB

Solution: Expected result: Boot disk is configured.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Configure boot disk with Debian Linux in the VM form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 11: Review identity and API access

- What to do:

Review service account settings; keep default or least-privilege

Solution: Expected result: Service account setting is reviewed.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Review service account and API access settings

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 12: Add labels

- What to do:

Add labels `purpose=practical` and `owner=student`

Solution: Expected result: Labels are added to the VM configuration.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Add purpose and owner labels in the VM form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 13: Review firewall options

- What to do:

Leave HTTP/HTTPS unchecked unless web server test required

Solution: Expected result: Firewall options are reviewed.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Review firewall options in the VM creation form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 14: Review advanced security options

- What to do:

Keep Shielded VM protections enabled

Solution: Expected result: Security options are reviewed.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Review Shielded VM security options

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 15: Create the VM

- What to do:

Review settings and click Create

Solution: Expected result: The VM appears in the VM instances list.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instancesAdd>

On screen: Tap Create button to provision the VM instance

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 16: Wait until VM is running

- What to do:

Wait until VM status becomes Running

Solution: Expected result: The VM is running.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Wait for VM status to show Running in the instances list

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 17: Observe VM details

- What to do:

Observe name, zone, internal IP, external IP, and status

Solution: Expected result: Basic VM details are identified.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: View VM details in the VM instances list

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 18: Connect to the VM using SSH

- What to do:

Click the SSH button next to the VM

Solution: Expected result: An SSH terminal opens.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Tap SSH button next to the VM instance

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 19: Run hostname command

- What to do:

Inside SSH run: `hostname`

Solution: Expected result: The VM hostname appears.

Command: `hostname`

STEP 20: Check operating system information

- What to do:

Run: `cat /etc/os-release`

Solution: Expected result: Operating system details are displayed.

Command: `cat /etc/os-release`

STEP 21: Check disk usage

- What to do:

Run: `df -h`

Solution: Expected result: Disk usage information appears.

Command: `df -h`

STEP 22: Check memory usage

- What to do:

Run: `free -h`

Solution: Expected result: Memory usage information appears.

Command: `free -h`

STEP 23: Check CPU information

- What to do:

Run: `lscpu`

Solution: Expected result: CPU information appears.

Command: `lscpu`

STEP 24: Update package information

- What to do:

Run: `sudo apt update`

Solution: Expected result: Package information is updated.

Command: `sudo apt update`

STEP 25: Optional web server test

- What to do:

If instructed: `sudo apt install -y nginx; systemctl status nginx`

Solution: Expected result: Nginx is installed and running if this optional step is performed.

Commands: `sudo apt install -y nginx; systemctl status nginx`

STEP 26: Exit SSH session

- What to do:

Run: `exit`

Solution: Expected result: SSH session is closed.

Command: `exit`

STEP 27: Stop the VM

- What to do:

Select the VM and click Stop; confirm if asked

Solution: Expected result: VM status changes to stopped or terminated.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Select VM and tap Stop button in Compute Engine

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 28: Start the VM again

- What to do:

Select stopped VM and click Start/Resume

Solution: Expected result: VM returns to running status.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Select stopped VM and tap Start button

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 29: Delete the VM after practical

- What to do:

Select the VM and click Delete; confirm deletion

Solution: Expected result: The VM is deleted from the VM instances list.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Select VM and tap Delete button to remove the instance

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

STEP 30: Check that no test VM remains

- What to do:

Refresh VM instances page and confirm test VM is gone

Solution: Expected result: No unnecessary VM instance remains.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Refresh VM instances list and confirm cleanup

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 2` on a machine with your GCP account to capture this step.

Practical 3: Create Buckets In Cloud Storage

Objective: In this practical, you will create, configure, upload to, inspect, and delete a Google Cloud Storage bucket using safe beginner-friendly settings.

STEP 1: Open Google Cloud Console

- What to do:

Open <https://console.cloud.google.com> and sign in

Solution: Expected result: Google Cloud Console opens successfully.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Open and sign in to Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 2: Select the correct project

- What to do:

Use project selector to select the practical project

Solution: Expected result: The correct project is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Select the correct project using the project selector

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 3: Open Cloud Storage

- What to do:

Search for Cloud Storage and open it

Solution: Expected result: The Cloud Storage page opens.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Search and open Cloud Storage in the console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 4: Open Buckets page

- What to do:

In Cloud Storage open Buckets

Solution: Expected result: The buckets list page opens.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Open Buckets page in Cloud Storage

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 5: Click Create

- What to do:

Click Create or Create bucket

Solution: Expected result: The bucket creation page opens.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Tap Create bucket button in Cloud Storage

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 6: Enter a globally unique bucket name

- What to do:

Enter name like `cloudnative-practical-3-yourname-2026`

Solution: Expected result: A valid unique bucket name is entered.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Enter a globally unique bucket name in the creation form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 7: Choose bucket location type

- What to do:

Choose Region and select us-central1 or instructed region

Solution: Expected result: Bucket location is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Select Region location type and region in bucket form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 8: Choose default storage class

- What to do:

Select Standard storage class

Solution: Expected result: Standard storage class is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Select Standard storage class in bucket form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 9: Configure access control

- What to do:

Choose Uniform bucket-level access

Solution: Expected result: Uniform bucket-level access is selected or enabled.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Enable Uniform bucket-level access in bucket form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 10: Configure public access prevention

- What to do:

Choose Enforce public access prevention

Solution: Expected result: Public access prevention is enforced or inherited.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Enable public access prevention in bucket form

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 11: Review object protection settings

- What to do:

Keep default soft delete protection or configure as instructed

Solution: Expected result: Object protection settings are reviewed.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Review soft delete object protection settings

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 12: Add labels

- What to do:

Add labels purpose=practical and owner=student

Solution: Expected result: Labels are added to the bucket.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Add purpose and owner labels to the bucket

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 13: Create the bucket

- What to do:

Review settings and click Create

Solution: Expected result: The bucket is created and appears in the bucket list.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Tap Create button to create the storage bucket

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 14: Open the bucket

- What to do:

Click the bucket name in the bucket list

Solution: Expected result: The bucket details page opens.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Open the bucket by clicking its name in the list

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 15: Create a small test file on your computer

- What to do:

Create cloudnative_test.txt with test content on your local computer

Solution: Expected result: A small test file is ready for upload.

File content: This is a test file for Cloud Storage Practical 3.

STEP 16: Upload the test file

- What to do:

Click Upload files and select cloudnative_test.txt

Solution: Expected result: The file appears in the object list.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Tap Upload files and upload cloudnative_test.txt to the bucket

[Screenshot pending] Run capture_gcp_practical_screenshots.py --login --practical 3 on a machine with your GCP account to capture this step.

STEP 17: Open object details

- What to do:

Click the uploaded object name and observe metadata

Solution: Expected result: Object metadata is visible.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Click uploaded object to view object details

[Screenshot pending] Run capture_gcp_practical_screenshots.py --login --practical 3 on a machine with your GCP account to capture this step.

STEP 18: Copy the object URI

- What to do:

Find and copy URI like `gs://bucket-name/cloudnative_test.txt`

Solution: Expected result: The object URI is copied or written in your observation table.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Copy the gs:// object URI from object details

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 19: Download the object

- What to do:

Use the download option to download the file

Solution: Expected result: The test file is downloaded successfully.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Download the object using the download option

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 20: Create a folder-like prefix

- What to do:

Click Create folder and name it practical-files

Solution: Expected result: A folder-like prefix appears in the bucket.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Tap Create folder and name it practical-files

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 21: Upload another test file into the folder

- What to do:

Open practical-files folder and upload notes.txt

Solution: Expected result: The second file appears under practical-files.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Upload notes.txt into the practical-files folder

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 22: Review permissions

- What to do:

Open Permissions tab; do not add public access

Solution: Expected result: Bucket permissions are reviewed and no public access is added.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Open Permissions tab and review bucket access settings

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 23: Review lifecycle option

- What to do:

Find lifecycle management options; do not create complex rules

Solution: Expected result: Lifecycle management option is identified.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Review lifecycle management options in bucket settings

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 24: Delete uploaded objects

- What to do:

Delete `cloudnative_test.txt`, `notes.txt`, and `practical-files` prefix

Solution: Expected result: The bucket becomes empty.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Delete uploaded objects and folder prefix from the bucket

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 25: Delete the bucket

- What to do:

Select bucket and click Delete; confirm by typing required text

Solution: Expected result: The bucket is deleted.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Select bucket and tap Delete button to remove it

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

STEP 26: Confirm cleanup

- What to do:

Refresh Buckets page and confirm practical bucket is gone

Solution: Expected result: No practical bucket remains in the bucket list.

Open in Google Cloud Console: <https://console.cloud.google.com/storage/browser>

On screen: Refresh Buckets page and confirm bucket deletion

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 3` on a machine with your GCP account to capture this step.

Practical 4: Create Users, Roles, And Permissions

Objective: In this practical, you will understand Google Cloud Identity and Access Management, add a principal to a project, assign a limited role, verify access, create a service account, understand custom roles, and remove access after the practical.

STEP 1: Open Google Cloud Console

- What to do:

Open <https://console.cloud.google.com> and sign in

Solution: Expected result: Google Cloud Console opens successfully.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Open and sign in to Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 2: Select the correct project

- What to do:

Use project selector to choose the practical project

Solution: Expected result: The correct project is selected.

Open in Google Cloud Console: <https://console.cloud.google.com>

On screen: Select the correct project using the project selector

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 3: Open IAM and Admin

- What to do:

Search for IAM and open IAM and Admin

Solution: Expected result: IAM and Admin page opens.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin>

On screen: Search and open IAM and Admin in the console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 4: Open the IAM page

- What to do:

In IAM and Admin click IAM

Solution: Expected result: The IAM policy list is visible.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Open IAM page showing principals and roles

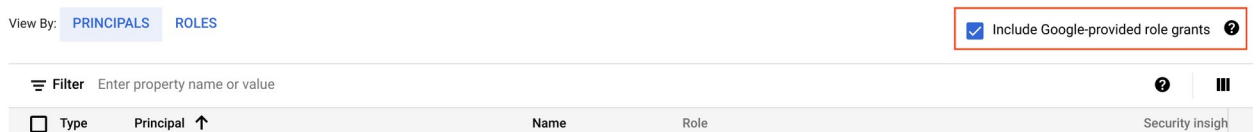


Figure: Official Google Cloud documentation screenshot for this screen. Run `capture_gcp_practical_screenshots.py --login --practical N` for a live capture.

STEP 5: Observe existing principals

- What to do:

Review principal names, types, assigned roles, and inherited roles

Solution: Expected result: Existing access entries are identified.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: View existing principals and roles on the IAM page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 6: Identify basic roles

- What to do:

Look for Owner, Editor, and Viewer roles

Solution: Expected result: You can identify whether basic roles exist in the project.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Identify basic roles in the IAM principals list

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 7: Plan the access to grant

- What to do:

Choose limited predefined role Storage Object Viewer for the lab

Solution: Expected result: The role to grant is selected.

STEP 8: Add a principal

- What to do:

Click Grant Access or Add principal; enter test user email

Solution: Expected result: The principal email is entered.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Tap Grant Access and enter test principal email

Edit access to "my-project"

Principal  my-user@example.com
Project my-project

Assign roles

Roles are composed of sets of permissions and determine what the principal can do with this resource. [Learn more](#)

Role  Logs Viewer	IAM condition (optional)  + ADD IAM CONDITION	
Access to view logs, except for logs with private contents.		
+ ADD ANOTHER ROLE		
SAVE	TEST CHANGES 	CANCEL

Figure: Official Google Cloud documentation screenshot for this screen. Run `capture_gcp_practical_screenshots.py --login --practical N` for a live capture.

STEP 9: Assign a predefined role

- What to do:

Search and select Cloud Storage > Storage Object Viewer

Solution: Expected result: The Storage Object Viewer role is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Assign Storage Object Viewer role to the principal

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 10: Avoid basic roles

- What to do:

Do not select Owner, Editor, or Viewer unless instructed

Solution: Expected result: No broad basic role is selected for the test principal.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Confirm no basic Owner or Editor role is selected

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 11: Optional condition for temporary access

- What to do:

Add IAM condition with expiry date if instructed by teacher

Solution: Expected result: Condition is added only if instructed.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Add optional IAM condition for temporary access if required

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 12: Save the role binding

- What to do:

Click Save to update the IAM allow policy

Solution: Expected result: The principal appears in the IAM list with the selected role.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Tap Save to apply the IAM role binding

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 13: Verify the assigned role

- What to do:

Search IAM list for principal; confirm Storage Object Viewer role

Solution: Expected result: The correct role is visible for the principal.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Verify Storage Object Viewer role on the IAM page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 14: Understand effective access

- What to do:

Check if principal has other direct or inherited roles

Solution: Expected result: You understand that multiple role bindings can combine to create broader access.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Review effective access including inherited roles on IAM page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 15: Open Roles page

- What to do:

In IAM and Admin open Roles

Solution: Expected result: The Roles page opens.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/roles>

On screen: Open Roles page in IAM and Admin

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 16: Search for a predefined role

- What to do:

Search for Storage Object Viewer and view role details

Solution: Expected result: Permissions included in the predefined role are visible.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/roles>

On screen: Search Storage Object Viewer role and view permissions

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 17: Observe permissions in the role

- What to do:

Read permissions listed in the role such as `storage.objects.get`

Solution: Expected result: You can identify that the role contains multiple permissions.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/roles>

On screen: View individual permissions inside the Storage Object Viewer role

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 18: Understand custom roles

- What to do:

Locate custom role creation option but do not create unless instructed

Solution: Expected result: You can identify where custom roles are managed.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/roles>

On screen: Locate custom roles section on the Roles page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 19: Open Service Accounts

- What to do:

In IAM and Admin open Service Accounts

Solution: Expected result: The Service Accounts page opens.

Open in Google Cloud Console:

<https://console.cloud.google.com/iam-admin/serviceaccounts>

On screen: Open Service Accounts page in IAM and Admin

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 20: Create a service account

- What to do:

Click Create service account; name `cloudnative-practical-sa`

Solution: Expected result: Service account basic details are entered.

Open in Google Cloud Console:

<https://console.cloud.google.com/iam-admin/serviceaccounts/create>

On screen: Tap Create service account and enter cloudnative-practical-sa details

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 21: Grant limited role to service account

- What to do:

Grant Storage Object Viewer or skip role granting as instructed

Solution: Expected result: The service account is created with limited or no project role.

Open in Google Cloud Console:

<https://console.cloud.google.com/iam-admin/serviceaccounts/create>

On screen: Grant limited role or skip during service account creation

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 22: Avoid creating service account keys

- What to do:

Do not create or download service account key file

Solution: Expected result: No unnecessary service account key is created.

Open in Google Cloud Console:

<https://console.cloud.google.com/iam-admin/serviceaccounts>

On screen: Skip service account key creation on the Keys tab

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 23: Review service account details

- What to do:

Review email, description, permissions, and Keys tab

Solution: Expected result: Service account information is reviewed.

Open in Google Cloud Console:

<https://console.cloud.google.com/iam-admin/serviceaccounts>

On screen: Review service account details and permissions page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 24: Remove test user access

- What to do:

Go to IAM, find test principal, remove Storage Object Viewer binding

Solution: Expected result: The test principal no longer has the lab role.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Remove test principal role binding on the IAM page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 25: Delete the service account

- What to do:

Select `cloudnative-practical-sa` and click Delete; confirm

Solution: Expected result: The test service account is deleted.

Open in Google Cloud Console:

<https://console.cloud.google.com/iam-admin/serviceaccounts>

On screen: Delete cloudnative-practical-sa service account

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 26: Review IAM recommendations if available

- What to do:

Search for IAM Recommender or check recommendations in console

Solution: Expected result: You can identify where IAM recommendations may appear.

Open in Google Cloud Console:

<https://console.cloud.google.com/iam-admin/recommender>

On screen: Open IAM Recommender in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 27: Review Policy Analyzer if available

- What to do:

Search for Policy Analyzer and open if available

Solution: Expected result: You can identify Policy Analyzer or understand its purpose.

Open in Google Cloud Console:

<https://console.cloud.google.com/iam-admin/analyzer>

On screen: Open Policy Analyzer in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

STEP 28: Final access check

- What to do:

Confirm test user access removed, service account deleted, no broad roles granted

Solution: Expected result: No unnecessary practical IAM access remains.

Open in Google Cloud Console: <https://console.cloud.google.com/iam-admin/iam>

On screen: Final IAM review confirming all test access is removed

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 4` on a machine with your GCP account to capture this step.

Practical 5: Create Docker Images And Containers And Run Containerized Applications

Objective: In this practical, you will create a simple application, write a Dockerfile, build a Docker image, run a Docker container, test the containerized application, inspect images and containers, stop/remove containers, and clean up Docker resources safely.

STEP 1: Create or open a Google Cloud VM

- What to do:

Use VM from Practical 2 or create small Debian/Ubuntu VM (e2-small, 10GB disk)

Solution: Expected result: A Linux VM is available and running.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Create or open a running Debian/Ubuntu VM in Compute Engine

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 5` on a machine with your GCP account to capture this step.

STEP 2: Connect to the VM using SSH

- What to do:

Open VM in console and click SSH button

Solution: Expected result: You are connected to the VM terminal.

Open in Google Cloud Console:

<https://console.cloud.google.com/compute/instances>

On screen: Tap SSH button to connect to the VM

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 5` on a machine with your GCP account to capture this step.

STEP 3: Update package information on the VM

- What to do:

Run: `sudo apt update`

Solution: Expected result: Package information is updated.

Command: `sudo apt update`

STEP 4: Install Docker Engine on the VM if needed

- What to do:

Install Docker Engine from official Debian repository using provided commands

Solution: Expected result: Docker Engine is installed on the VM.

Commands: `sudo apt install -y ca-certificates curl; install Docker repo; sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`

STEP 5: Check Docker installation

- What to do:

Run: `docker --version`

Solution: Expected result: Docker version is displayed.

Command: `docker --version`

STEP 6: Check Docker is running

- What to do:

Run: `sudo docker info`

Solution: Expected result: Docker system information is displayed.

Command: `sudo docker info`

STEP 7: Optional permission setup for lab convenience

- What to do:

Run: `sudo usermod -aG docker $USER`; exit and reconnect SSH

Solution: Expected result: After reconnecting, Docker commands can run without `sudo`.

Command: `sudo usermod -aG docker $USER`

STEP 8: Test Docker with hello-world

- What to do:

Run: `sudo docker run hello-world`

Solution: Expected result: Docker prints a hello-world success message.

Command: `sudo docker run hello-world`

STEP 9: Create a practical folder

- What to do:

Run: `mkdir cloudnative-docker-practical && cd cloudnative-docker-practical`

Solution: Expected result: You are inside the practical folder.

Commands: `mkdir cloudnative-docker-practical; cd cloudnative-docker-practical`

STEP 10: Create package.json

- What to do:

Create `package.json` with Node.js app metadata

Solution: Expected result: `package.json` is created.

STEP 11: Create server.js

- What to do:

Create server.js with simple HTTP server on port 3000

Solution: Expected result: server.js is created.

STEP 12: Create .dockerignore

- What to do:

Create .dockerignore excluding .git, node_modules, .env, keys

Solution: Expected result: .dockerignore is created.

STEP 13: Create Dockerfile

- What to do:

Create Dockerfile with node:22-alpine, non-root user, EXPOSE 3000

Solution: Expected result: Dockerfile is created.

STEP 14: Understand the Dockerfile instructions

- What to do:

Review FROM, WORKDIR, COPY, RUN, USER, EXPOSE, CMD purposes

Solution: Expected result: You can explain the purpose of each Dockerfile instruction.

STEP 15: Build the Docker image

- What to do:

Run: `sudo docker build -t cloudfnative-practical-5:1.0 .`

Solution: Expected result: Docker image is built successfully.

Command: `sudo docker build -t cloudfnative-practical-5:1.0 .`

STEP 16: List Docker images

- What to do:

Run: `sudo docker images`

Solution: Expected result: cloudnative-practical-5 with tag 1.0 appears.

Command: `sudo docker images`

STEP 17: Run the container

- What to do:

Run: `sudo docker run --name cloudnative-app -p 8080:3000 cloudnative-practical-5:1.0`

Solution: Expected result: Application starts with message 'Server running on port 3000'.

Command: `sudo docker run --name cloudnative-app -p 8080:3000 cloudnative-practical-5:1.0`

STEP 18: Test the application from inside the VM

- What to do:

In second SSH session run: `curl http://localhost:8080`

Solution: Expected result: Output: Hello from Cloud Native Docker Practical 5

Command: `curl http://localhost:8080`

STEP 19: Optional browser test from local machine

- What to do:

If instructed, open firewall port 8080 and browse `http://VM_EXTERNAL_IP:8080`

Solution: Expected result: Browser displays application message only if firewall access is allowed.

STEP 20: Stop the running container

- What to do:

Press Ctrl+C in the terminal running the container

Solution: Expected result: The container stops.

STEP 21: Run container in detached mode

- What to do:

Run: `sudo docker run -d --name cloudnative-app -p 8080:3000 cloudnative-practical-5:1.0`

Solution: Expected result: Docker prints container ID and returns control.

Command: `sudo docker run -d --name cloudnative-app -p 8080:3000 cloudnative-practical-5:1.0`

STEP 22: List running containers

- What to do:

Run: `sudo docker ps`

Solution: Expected result: cloudnative-app appears in running container list.

Command: `sudo docker ps`

STEP 23: View container logs

- What to do:

Run: `sudo docker logs cloudnative-app`

Solution: Expected result: Server startup log appears.

Command: `sudo docker logs cloudnative-app`

STEP 24: Execute a command inside the container

- What to do:

Run: `sudo docker exec cloudnative-app node --version`

Solution: Expected result: Node.js version is displayed.

Command: `sudo docker exec cloudnative-app node --version`

STEP 25: Inspect container configuration

- What to do:

Run: `sudo docker inspect cloudnative-app`

Solution: Expected result: Container configuration details displayed in JSON.

Command: `sudo docker inspect cloudnative-app`

STEP 26: Stop the detached container

- What to do:

Run: `sudo docker stop cloudnative-app`

Solution: Expected result: The container stops.

Command: `sudo docker stop cloudnative-app`

STEP 27: List all containers

- What to do:

Run: `sudo docker ps -a`

Solution: Expected result: cloudnative-app appears as stopped container.

Command: `sudo docker ps -a`

STEP 28: Remove the stopped container

- What to do:

Run: `sudo docker rm cloudnative-app`

Solution: Expected result: Stopped container is removed.

Command: `sudo docker rm cloudnative-app`

STEP 29: Run container with automatic cleanup

- What to do:

Run with `--rm` flag, test with `curl`, then `Ctrl+C`

Solution: Expected result: App runs and container is removed automatically after stopping.

Command: `sudo docker run --rm --name cloudnative-app-temp -p 8080:3000 cloudnative-practical-5:1.0`

STEP 30: Tag the image with another tag

- What to do:

Run: `sudo docker tag cloudnative-practical-5:1.0 cloudnative-practical-5:latest`

Solution: Expected result: Image has both 1.0 and latest tags.

Command: `sudo docker tag cloudnative-practical-5:1.0 cloudnative-practical-5:latest`

STEP 31: Check image history

- What to do:

Run: `sudo docker history cloudnative-practical-5:1.0`

Solution: Expected result: Image layer history is displayed.

Command: `sudo docker history cloudnative-practical-5:1.0`

STEP 32: Optional image scan

- What to do:

Run: `sudo docker scout quickview` or `trivy image` if available

Solution: Expected result: Scanner output appears if tool is installed.

STEP 33: Remove image tags

- What to do:

Run: `sudo docker rmi cloudnative-practical-5:latest and :1.0`

Solution: Expected result: Image tags are removed.

Commands: `sudo docker rmi cloudnative-practical-5:latest; sudo docker rmi cloudnative-practical-5:1.0`

STEP 34: Check Docker disk usage

- What to do:

Run: `sudo docker system df`

Solution: Expected result: Docker disk usage summary appears.

Command: `sudo docker system df`

STEP 35: Clean up carefully

- What to do:

If allowed run: `sudo docker builder prune`

Solution: Expected result: Unused build cache is cleaned if confirmed.

Command: `sudo docker builder prune`

Practical 6: Create A Kubernetes Cluster And Deploy Applications Using Pods And Services

Objective: In this practical, you will create a Google Kubernetes Engine cluster, connect to it using kubectl, deploy a containerized application, observe Pods, expose the application using a Kubernetes Service, test the application safely, and clean up all resources after the practical.

STEP 1: Open Google Cloud Console

- What to do:

Open <https://console.cloud.google.com> and sign in

Solution: Expected result: Google Cloud Console opens successfully.

Open in Google Cloud Console: <https://console.cloud.google.com/kubernetes/list>

On screen: Open and sign in to Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 6` on a machine with your GCP account to capture this step.

STEP 2: Select the correct project

- What to do:

Use project selector for the practical project

Solution: Expected result: The correct project is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/kubernetes/list>

On screen: Select the correct project using the project selector

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 6` on a machine with your GCP account to capture this step.

STEP 3: Open Cloud Shell

- What to do:

Click Cloud Shell icon in the console

Solution: Expected result: Cloud Shell terminal opens.

Open in Google Cloud Console: <https://console.cloud.google.com/kubernetes/list>

On screen: Click Cloud Shell icon in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 6` on a machine with your GCP account to capture this step.

STEP 4: Check active account and project

- What to do:

Run: `gcloud auth list; gcloud config get-value project`

Solution: Expected result: Correct account and project displayed.

Commands: `gcloud auth list; gcloud config get-value project`

STEP 5: Enable required APIs if needed

- What to do:

Run: `gcloud services enable container.googleapis.com`

Solution: Expected result: Kubernetes Engine API is enabled.

Command: `gcloud services enable container.googleapis.com`

STEP 6: Set default region

- What to do:

Run: `gcloud config set compute/region us-central1`

Solution: Expected result: Default compute region is set.

Command: `gcloud config set compute/region us-central1`

STEP 7: Create a GKE Autopilot cluster

- What to do:

Run: `gcloud container clusters create-auto cloudnative-cluster-1 --location=us-central1`

Solution: Expected result: Cluster creation starts; may take several minutes.

Command: `gcloud container clusters create-auto cloudnative-cluster-1 --location=us-central1`

STEP 8: Verify cluster creation

- What to do:

Run: `gcloud container clusters list`

Solution: Expected result: `cloudnative-cluster-1` appears in cluster list.

Command: `gcloud container clusters list`

STEP 9: Connect kubectl to the cluster

- What to do:

Run: `gcloud container clusters get-credentials cloudnative-cluster-1 --location=us-central1`

Solution: Expected result: `kubectl` context configured for cluster.

Command: `gcloud container clusters get-credentials cloudnative-cluster-1 --location=us-central1`

STEP 10: Check cluster nodes

- What to do:

Run: `kubectl get nodes`

Solution: Expected result: Cluster node information displayed.

Command: `kubectl get nodes`

STEP 11: Check current Kubernetes resources

- What to do:

Run: `kubectl get pods; kubectl get deployments; kubectl get services`

Solution: Expected result: No application resources yet except default Service.

Commands: `kubectl get pods; kubectl get deployments; kubectl get services`

STEP 12: Create a Deployment

- What to do:

Run: `kubectl create deployment hello-server --image=us-docker.pkg.dev/google-samples/containers/gke/hello-app:1.0`

Solution: Expected result: Deployment hello-server created.

Command: `kubectl create deployment hello-server`

`--image=us-docker.pkg.dev/google-samples/containers/gke/hello-app:1.0`

STEP 13: View the Deployment

- What to do:

Run: `kubectl get deployments`

Solution: Expected result: hello-server appears in Deployment list.

Command: `kubectl get deployments`

STEP 14: View the Pods

- What to do:

Run: `kubectl get pods`

Solution: Expected result: Pod for hello-server reaches Running status.

Command: `kubectl get pods`

STEP 15: Describe the Pod

- What to do:

Run: `kubectl describe pod POD_NAME`

Solution: Expected result: Detailed Pod information displayed.

Command: `kubectl describe pod POD_NAME`

STEP 16: View application logs

- What to do:

Run: `kubectl logs deployment/hello-server`

Solution: Expected result: Application logs displayed.

Command: `kubectl logs deployment/hello-server`

STEP 17: Expose the Deployment using a ClusterIP Service

- What to do:

Run: `kubectl expose deployment hello-server --type=ClusterIP --port=80 --target-port=8080`

Solution: Expected result: Service hello-server created.

Command: `kubectl expose deployment hello-server --type=ClusterIP --port=80 --target-port=8080`

STEP 18: View Services

- What to do:

Run: `kubectl get services`

Solution: Expected result: hello-server appears as ClusterIP Service.

Command: `kubectl get services`

STEP 19: Test the application using port-forward

- What to do:

Run: `kubectl port-forward service/hello-server 8080:80; curl http://localhost:8080`

Solution: Expected result: hello-app response displayed.

Commands: `kubectl port-forward service/hello-server 8080:80; curl http://localhost:8080`

STEP 20: Stop port-forward

- What to do:

Press Ctrl+C

Solution: Expected result: Port-forwarding stops.

STEP 21: Scale the Deployment

- What to do:

Run: `kubectl scale deployment hello-server --replicas=3`

Solution: Expected result: Deployment updated to three replicas.

Command: `kubectl scale deployment hello-server --replicas=3`

STEP 22: View scaled Pods

- What to do:

Run: `kubectl get pods`

Solution: Expected result: Three hello-server Pods appear.

Command: `kubectl get pods`

STEP 23: Understand self-healing

- What to do:

Delete one Pod with `kubectl delete pod POD_NAME`; then `kubectl get pods`

Solution: Expected result: Replacement Pod created automatically.

Commands: `kubectl delete pod POD_NAME`; `kubectl get pods`

STEP 24: View ReplicaSet

- What to do:

Run: `kubectl get replicaset`

Solution: Expected result: ReplicaSet for hello-server displayed.

Command: `kubectl get replicaset`

STEP 25: Optional public LoadBalancer Service

- What to do:

If allowed: `kubectl delete service hello-server`; `kubectl expose deployment hello-server --type=LoadBalancer --port=80 --target-port=8080`

Solution: Expected result: LoadBalancer Service created.

Commands: `kubectl delete service hello-server`; `kubectl expose deployment hello-server --type=LoadBalancer --port=80 --target-port=8080`

STEP 26: Optional test using external IP

- What to do:

Run: `kubectl get service hello-server`; open `http://EXTERNAL_IP`

Solution: Expected result: Application response appears in browser.

Commands: `kubectl get service hello-server`; browse `http://EXTERNAL_IP`

STEP 27: Delete the Service

- What to do:

Run: `kubectl delete service hello-server`

Solution: Expected result: Service hello-server deleted.

Command: `kubectl delete service hello-server`

STEP 28: Delete the Deployment

- What to do:

Run: `kubectl delete deployment hello-server`

Solution: Expected result: Deployment and Pods deleted.

Command: `kubectl delete deployment hello-server`

STEP 29: Confirm application cleanup

- What to do:

Run: `kubectl get deployments; kubectl get pods; kubectl get services`

Solution: Expected result: hello-server resources no longer appear.

Commands: `kubectl get deployments; kubectl get pods; kubectl get services`

STEP 30: Delete the GKE cluster

- What to do:

Run: `gcloud container clusters delete cloudnative-cluster-1 --location=us-central1`

Solution: Expected result: Cluster deletion starts and completes.

Command: `gcloud container clusters delete cloudnative-cluster-1 --location=us-central1`

STEP 31: Confirm cluster deletion

- What to do:

Run: `gcloud container clusters list`

Solution: Expected result: cloudnative-cluster-1 is not listed.

Command: `gcloud container clusters list`

Practical 7: Design And Deploy Microservices Using Cloud Run

Objective: In this practical, you will design a simple microservices-based application and deploy it on Google Cloud Run. You will create two services: a public frontend service and a private backend service.

STEP 1: Open Google Cloud Console

- What to do:

Open <https://console.cloud.google.com> and sign in

Solution: Expected result: Google Cloud Console opens successfully.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Open and sign in to Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 7` on a machine with your GCP account to capture this step.

STEP 2: Select the correct project

- What to do:

Use project selector for the practical project

Solution: Expected result: The correct project is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Select the correct project using the project selector

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 7` on a machine with your GCP account to capture this step.

STEP 3: Open Cloud Shell

- What to do:

Open Cloud Shell from Google Cloud Console

Solution: Expected result: Cloud Shell opens.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Click Cloud Shell icon in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 7` on a machine with your GCP account to capture this step.

STEP 4: Check active account and project

- What to do:

Run: `gcloud auth list; gcloud config get-value project`

Solution: Expected result: Correct account and project displayed.

Commands: `gcloud auth list; gcloud config get-value project`

STEP 5: Set region variable

- What to do:

Run: `export REGION=us-central1; export PROJECT_ID=$(gcloud config get-value project)`

Solution: Expected result: Region and project variables set.

Commands: `export REGION=us-central1; export PROJECT_ID=$(gcloud config get-value project)`

STEP 6: Enable required APIs

- What to do:

Run: `gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com`

Solution: Expected result: Required APIs enabled.

Commands: `gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com`

STEP 7: Create practical folder

- What to do:

Run: `mkdir cloudfnative-microservices-practical && cd cloudfnative-microservices-practical`

Solution: Expected result: Inside practical folder.

Commands: `mkdir cloudfnative-microservices-practical; cd cloudfnative-microservices-practical`

STEP 8: Create backend service folder

- What to do:

Run: `mkdir backend && cd backend`

Solution: Expected result: backend folder created and opened.

Commands: `mkdir backend; cd backend`

STEP 9: Create backend package.json

- What to do:

Create package.json for backend Node.js service

Solution: Expected result: Backend package.json created.

STEP 10: Create backend server.js

- What to do:

Create server.js returning JSON with /health endpoint on PORT

Solution: Expected result: Backend server.js created.

STEP 11: Create backend Dockerfile

- What to do:

Create Dockerfile with node:22-alpine and non-root user

Solution: Expected result: Backend Dockerfile created.

STEP 12: Create backend .dockerignore

- What to do:

Create .dockerignore excluding .git, node_modules, .env, keys

Solution: Expected result: Backend .dockerignore created.

STEP 13: Deploy backend as a private Cloud Run service

- What to do:

Run: `gcloud run deploy cloudnative-backend --source . --region $REGION --no-allow-unauthenticated`

Solution: Expected result: Backend built and deployed as private service.

Command: `gcloud run deploy cloudnative-backend --source . --region $REGION --no-allow-unauthenticated`

STEP 14: Save backend URL

- What to do:

Run: `export BACKEND_URL=$(gcloud run services describe cloudnative-backend --region $REGION --format='value(status.url)'); echo $BACKEND_URL`

Solution: Expected result: Backend URL displayed and stored.

Commands: `export BACKEND_URL=...; echo $BACKEND_URL`

STEP 15: Return to main folder

- What to do:

Run: `cd ..`

Solution: Expected result: Back in cloudnative-microservices-practical.
Command: `cd ..`

STEP 16: Create frontend folder

- What to do:

Run: `mkdir frontend && cd frontend`

Solution: Expected result: frontend folder created.
Commands: `mkdir frontend; cd frontend`

STEP 17: Create frontend package.json

- What to do:

Create package.json with google-auth-library dependency

Solution: Expected result: Frontend package.json created.

STEP 18: Create frontend server.js

- What to do:

Create server.js calling private backend with identity token

Solution: Expected result: Frontend server.js created.

STEP 19: Create frontend Dockerfile

- What to do:

Create Dockerfile with node:22-alpine and non-root user

Solution: Expected result: Frontend Dockerfile created.

STEP 20: Create frontend .dockerignore

- What to do:

Create .dockerignore excluding sensitive files

Solution: Expected result: Frontend .dockerignore created.

STEP 21: Create a dedicated frontend service account

- What to do:

Run: `gcloud iam service-accounts create cloudnative-frontend-sa --display-name='Cloud Native Frontend Service Account'`

Solution: Expected result: Frontend service account created.

Command: `gcloud iam service-accounts create cloudnative-frontend-sa`

STEP 22: Store frontend service account email

- What to do:

Run: `export FRONTEND_SA=cloudnative-frontend-sa@$PROJECT_ID.iam.gserviceaccount.com; echo $FRONTEND_SA`

Solution: Expected result: Service account email displayed.

Commands: `export FRONTEND_SA=...; echo $FRONTEND_SA`

STEP 23: Grant frontend permission to invoke backend

- What to do:

Run: `gcloud run services add-iam-policy-binding cloudnative-backend --region $REGION --member=serviceAccount:$FRONTEND_SA --role=roles/run.invoker`

Solution: Expected result: Frontend SA receives Cloud Run Invoker on backend.

Command: `gcloud run services add-iam-policy-binding cloudnative-backend --member=serviceAccount:$FRONTEND_SA --role=roles/run.invoker`

STEP 24: Deploy frontend as a public Cloud Run service

- What to do:

Run: `gcloud run deploy cloudnative-frontend --source . --region $REGION --service-account $FRONTEND_SA --set-env-vars BACKEND_URL=$BACKEND_URL --allow-unauthenticated`

Solution: Expected result: Frontend built and deployed publicly.

Command: `gcloud run deploy cloudnative-frontend --source . --region $REGION --service-account $FRONTEND_SA --set-env-vars BACKEND_URL=$BACKEND_URL --allow-unauthenticated`

STEP 25: Save frontend URL

- What to do:

Run: `export FRONTEND_URL=$(gcloud run services describe cloudnative-frontend --region $REGION --format='value(status.url)'); echo $FRONTEND_URL`

Solution: Expected result: Frontend URL displayed.

Commands: `export FRONTEND_URL=...; echo $FRONTEND_URL`

STEP 26: Test frontend service

- What to do:

Open frontend URL in browser or run: `curl $FRONTEND_URL`

Solution: Expected result: Response shows frontend received response from backend.

Command: `curl $FRONTEND_URL`

STEP 27: Try to access backend directly

- What to do:

Run: `curl $BACKEND_URL`

Solution: Expected result: Direct backend call rejected without authentication.
Command: `curl $BACKEND_URL`

STEP 28: View Cloud Run services

- What to do:

Run: `gcloud run services list --region $REGION`

Solution: Expected result: cloudnative-backend and cloudnative-frontend listed.
Command: `gcloud run services list --region $REGION`

STEP 29: View frontend logs

- What to do:

Run: `gcloud run services logs read cloudnative-frontend --region $REGION --limit=20`

Solution: Expected result: Recent frontend logs displayed.
Command: `gcloud run services logs read cloudnative-frontend --region $REGION --limit=20`

STEP 30: View backend logs

- What to do:

Run: `gcloud run services logs read cloudnative-backend --region $REGION --limit=20`

Solution: Expected result: Recent backend logs displayed.
Command: `gcloud run services logs read cloudnative-backend --region $REGION --limit=20`

STEP 31: Review service configuration

- What to do:

Run: `gcloud run services describe cloudnative-frontend and cloudnative-backend --region $REGION`

Solution: Expected result: Service configuration displayed.

Commands: `gcloud run services describe cloudnative-frontend`; `gcloud run services describe cloudnative-backend`

STEP 32: Delete frontend service

- What to do:

Run: `gcloud run services delete cloudnative-frontend --region $REGION`

Solution: Expected result: Frontend service deleted.

Command: `gcloud run services delete cloudnative-frontend --region $REGION`

STEP 33: Delete backend service

- What to do:

Run: `gcloud run services delete cloudnative-backend --region $REGION`

Solution: Expected result: Backend service deleted.

Command: `gcloud run services delete cloudnative-backend --region $REGION`

STEP 34: Delete service account

- What to do:

Run: `gcloud iam service-accounts delete $FRONTEND_SA`

Solution: Expected result: Frontend service account deleted.

Command: `gcloud iam service-accounts delete $FRONTEND_SA`

STEP 35: Confirm cleanup

- What to do:

Run: `gcloud run services list --region $REGION`

Solution: Expected result: Both services no longer appear.
Command: `gcloud run services list --region $REGION`

STEP 36: Optional cleanup of build images

- What to do:

Open Artifact Registry in console and delete practical-related images only

Solution: Expected result: Practical-related unused images removed if required.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Open Artifact Registry and delete practical build images

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 7` on a machine with your GCP account to capture this step.

Practical 8: Automate Build And Deployment Pipeline

Objective: In this practical, you will automate the build and deployment of a containerized application using Google Cloud Build, Artifact Registry, and Cloud Run.

STEP 1: Open Google Cloud Console

- What to do:

Open <https://console.cloud.google.com> and sign in

Solution: Expected result: Google Cloud Console opens successfully.

Open in Google Cloud Console: <https://console.cloud.google.com/cloud-build/builds>

On screen: Open and sign in to Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 8` on a machine with your GCP account to capture this step.

STEP 2: Select the correct project

- What to do:

Use project selector for the practical project

Solution: Expected result: The correct project is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/cloud-build/builds>

On screen: Select the correct project using the project selector

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 8` on a machine with your GCP account to capture this step.

STEP 3: Open Cloud Shell

- What to do:

Open Cloud Shell from Google Cloud Console

Solution: Expected result: Cloud Shell terminal opens.

Open in Google Cloud Console: <https://console.cloud.google.com/cloud-build/builds>

On screen: Click Cloud Shell icon in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 8` on a machine with your GCP account to capture this step.

STEP 4: Check active account and project

- What to do:

Run: `gcloud auth list; gcloud config get-value project`

Solution: Expected result: Correct account and project displayed.

Commands: `gcloud auth list; gcloud config get-value project`

STEP 5: Set environment variables

- What to do:

Run: `export PROJECT_ID, REGION, REPOSITORY, SERVICE` variables

Solution: Expected result: Variables set.

Commands: `export PROJECT_ID=$(gcloud config get-value project); export REGION=us-central1; export REPOSITORY=cloudnative-pipeline-repo; export SERVICE=cloudnative-pipeline-app`

STEP 6: Enable required APIs

- What to do:

Run: `gcloud services enable cloudbuild.googleapis.com run.googleapis.com artifactregistry.googleapis.com`

Solution: Expected result: Required APIs enabled.

Commands: `gcloud services enable cloudbuild.googleapis.com run.googleapis.com artifactregistry.googleapis.com`

STEP 7: Create an Artifact Registry repository

- What to do:

Run: `gcloud artifacts repositories create $REPOSITORY --repository-format=docker --location=$REGION`

Solution: Expected result: Docker repository created.

Command: `gcloud artifacts repositories create $REPOSITORY --repository-format=docker --location=$REGION`

STEP 8: Create a runtime service account for Cloud Run

- What to do:

Run: `gcloud iam service-accounts create cloudrun-practical-runtime-sa`

Solution: Expected result: Runtime service account created.

Command: `gcloud iam service-accounts create cloudrun-practical-runtime-sa`

STEP 9: Store runtime service account email

- What to do:

Run: `export RUNTIME_SA=cloudrun-practical-runtime-sa@$PROJECT_ID.iam.gserviceaccount.com; echo $RUNTIME_SA`

Solution: Expected result: Runtime SA email displayed.

Commands: `export RUNTIME_SA=...; echo $RUNTIME_SA`

STEP 10: Create a Cloud Build service account

- What to do:

Run: `gcloud iam service-accounts create cloudbuild-practical-sa`

Solution: Expected result: Build service account created.

Command: `gcloud iam service-accounts create cloudbuild-practical-sa`

STEP 11: Store build service account email

- What to do:

Run: `export BUILD_SA=cloudbuild-practical-sa@$PROJECT_ID.iam.gserviceaccount.com; echo $BUILD_SA`

Solution: Expected result: Build SA email displayed.

Commands: `export BUILD_SA=...; echo $BUILD_SA`

STEP 12: Grant required roles to build service account

- What to do:

Grant roles/run.admin, artifactregistry.writer, logging.logWriter, and iam.serviceAccountUser

Solution: Expected result: Required IAM bindings added.

Commands: `gcloud projects add-iam-policy-binding ... roles/run.admin; roles/artifactregistry.writer; roles/logging.logWriter; iam.serviceAccountUser on RUNTIME_SA`

STEP 13: Create application folder

- What to do:

Run: `mkdir cloudnative-pipeline-app && cd cloudnative-pipeline-app`

Solution: Expected result: Application folder created.

Commands: `mkdir cloudnative-pipeline-app; cd cloudnative-pipeline-app`

STEP 14: Create package.json

- What to do:

Create package.json with start and test scripts

Solution: Expected result: package.json created.

STEP 15: Create server.js

- What to do:

Create server.js with /health endpoint listening on PORT

Solution: Expected result: server.js created.

STEP 16: Create a simple test file

- What to do:

Create test.js checking server.js exists

Solution: Expected result: test.js created.

STEP 17: Create Dockerfile

- What to do:

Create Dockerfile with node:22-alpine and non-root user

Solution: Expected result: Dockerfile created.

STEP 18: Create .dockerignore

- What to do:

Create .dockerignore excluding sensitive files

Solution: Expected result: .dockerignore created.

STEP 19: Create cloudbuild.yaml

- What to do:

Create cloudbuild.yaml with test, build, push, and deploy steps

Solution: Expected result: cloudbuild.yaml created with substitutions _REGION, _REPOSITORY, _SERVICE.

STEP 20: Confirm runtime service account variable

- What to do:

Run: `echo $RUNTIME_SA`

Solution: Expected result: Runtime SA email displayed.

Command: `echo $RUNTIME_SA`

STEP 21: Review cloudbuild.yaml

- What to do:

Run: `cat cloudbuild.yaml`

Solution: Expected result: Cloud Build configuration displayed.

Command: `cat cloudbuild.yaml`

STEP 22: Run the pipeline manually

- What to do:

Run: `gcloud builds submit --config=cloudbuild.yaml --service-account=... --substitutions=...`

Solution: Expected result: Build runs test, builds image, pushes to Artifact Registry, deploys to Cloud Run.

Command: `gcloud builds submit --config=cloudbuild.yaml --service-account=projects/$PROJECT_ID/serviceAccounts/$BUILD_SA --substitutions=_REGION=$REGION,_REPOSITORY=$REPOSITORY,_SERVICE=$SERVICE,_RUNTIME_SA=$RUNTIME_SA`

STEP 23: View build history

- What to do:

Run: `gcloud builds list --limit=5`

Solution: Expected result: Recent builds listed.

Command: `gcloud builds list --limit=5`

STEP 24: Get Cloud Run service URL

- What to do:

Run: `export SERVICE_URL=$(gcloud run services describe $SERVICE --region=$REGION --format='value(status.url)'); echo $SERVICE_URL`

Solution: Expected result: Cloud Run service URL displayed.

Commands: `export SERVICE_URL=...; echo $SERVICE_URL`

STEP 25: Test the deployed application

- What to do:

Run: `curl $SERVICE_URL`

Solution: Expected result: Hello from automated Cloud Build and Cloud Run pipeline

Command: `curl $SERVICE_URL`

STEP 26: View Cloud Run service

- What to do:

Run: `gcloud run services list --region=$REGION`

Solution: Expected result: `cloudnative-pipeline-app` appears.

Command: `gcloud run services list --region=$REGION`

STEP 27: View Artifact Registry images

- What to do:

Run: `gcloud artifacts docker images list`
`$REGION-docker.pkg.dev/$PROJECT_ID/$REPOSITORY`

Solution: Expected result: Application image appears.

Command: `gcloud artifacts docker images list ...`

STEP 28: View Cloud Run logs

- What to do:

Run: `gcloud run services logs read $SERVICE --region=$REGION --limit=20`

Solution: Expected result: Recent logs displayed.

Command: `gcloud run services logs read $SERVICE --region=$REGION --limit=20`

STEP 29: Optional repository trigger concept

- What to do:

Review how to connect Git repo and create Cloud Build trigger

Solution: Expected result: You can see how manual pipeline becomes automated through a trigger.

STEP 30: Optional pipeline failure test

- What to do:

Edit `test.js` to fail with `process.exit(1)`; rerun build; undo after

Solution: Expected result: Cloud Build fails during test step and does not deploy.

STEP 31: Delete Cloud Run service

- What to do:

Run: `gcloud run services delete $SERVICE --region=$REGION`

Solution: Expected result: Cloud Run service deleted.

Command: `gcloud run services delete $SERVICE --region=$REGION`

STEP 32: Delete Artifact Registry repository

- What to do:

Run: `gcloud artifacts repositories delete $REPOSITORY --location=$REGION`

Solution: Expected result: Artifact Registry repository deleted.

Command: `gcloud artifacts repositories delete $REPOSITORY --location=$REGION`

STEP 33: Remove IAM bindings and service accounts

- What to do:

Remove IAM bindings and delete BUILD_SA and RUNTIME_SA

Solution: Expected result: Practical IAM bindings removed and service accounts deleted.

Commands: `gcloud projects remove-iam-policy-binding ...`; `gcloud iam service-accounts delete $BUILD_SA`; `gcloud iam service-accounts delete $RUNTIME_SA`

STEP 34: Confirm cleanup

- What to do:

Run: `gcloud run services list`; `gcloud artifacts repositories list`

Solution: Expected result: Practical service and repository no longer appear.

Commands: `gcloud run services list --region=$REGION`; `gcloud artifacts repositories list --location=$REGION`

Practical 9: Use Cloud Monitoring And Logging Tools

Objective: In this practical, you will use Google Cloud Monitoring and Cloud Logging to observe a running cloud-native application. You will deploy a small Cloud Run application, generate requests and logs, view logs in Logs Explorer, view metrics in Metrics Explorer, understand dashboards, create or review a log-based metric, understand alerting and uptime checks, and clean up resources after the practical.

STEP 1: Open Google Cloud Console

- What to do:

Open <https://console.cloud.google.com> and sign in

Solution: Expected result: Google Cloud Console opens successfully.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Open and sign in to Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 2: Select the correct project

- What to do:

Use project selector for the practical project

Solution: Expected result: The correct project is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Select the correct project using the project selector

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 3: Open Cloud Shell

- What to do:

Open Cloud Shell from Google Cloud Console

Solution: Expected result: Cloud Shell terminal opens.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Click Cloud Shell icon in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 4: Check active account and project

- What to do:

Run: `gcloud auth list; gcloud config get-value project`

Solution: Expected result: Correct account and project displayed.

Commands: `gcloud auth list; gcloud config get-value project`

STEP 5: Set environment variables

- What to do:

Run: `export PROJECT_ID, REGION, SERVICE` variables

Solution: Expected result: Variables set.

Commands: `export PROJECT_ID=$(gcloud config get-value project); export REGION=us-central1; export SERVICE=cloudnative-observability-app`

STEP 6: Enable required APIs

- What to do:

Run: `gcloud services enable run.googleapis.com cloudbuild.googleapis.com logging.googleapis.com monitoring.googleapis.com`

Solution: Expected result: Required APIs enabled.

Commands: `gcloud services enable run.googleapis.com cloudbuild.googleapis.com logging.googleapis.com monitoring.googleapis.com`

STEP 7: Create application folder

- What to do:

Run: `mkdir cloudnative-observability-app && cd cloudnative-observability-app`

Solution: Expected result: Application folder created.

Commands: `mkdir cloudnative-observability-app; cd cloudnative-observability-app`

STEP 8: Create package.json

- What to do:

Create package.json for Node.js observability app

Solution: Expected result: package.json created.

STEP 9: Create server.js with structured logs

- What to do:

Create server.js writing JSON logs with /health and /error endpoints

Solution: Expected result: server.js created with structured JSON logging.

STEP 10: Create Dockerfile

- What to do:

Create Dockerfile with node:22-alpine and non-root user

Solution: Expected result: Dockerfile created.

STEP 11: Create .dockerignore

- What to do:

Create .dockerignore excluding sensitive files

Solution: Expected result: .dockerignore created.

STEP 12: Deploy the application to Cloud Run

- What to do:

Run: `gcloud run deploy $SERVICE --source . --region $REGION --allow-unauthenticated`

Solution: Expected result: Cloud Run service deployed.

Command: `gcloud run deploy $SERVICE --source . --region $REGION --allow-unauthenticated`

STEP 13: Save service URL

- What to do:

Run: `export SERVICE_URL=$(gcloud run services describe $SERVICE --region $REGION --format='value(status.url)'); echo $SERVICE_URL`

Solution: Expected result: Service URL displayed.

Commands: `export SERVICE_URL=...; echo $SERVICE_URL`

STEP 14: Generate successful requests

- What to do:

Run: `for i in 1 2 3 4 5; do curl $SERVICE_URL; done`

Solution: Expected result: Application response appears several times.

Command: `for i in 1 2 3 4 5; do curl $SERVICE_URL; done`

STEP 15: Generate health check request

- What to do:

Run: `curl $SERVICE_URL/health`

Solution: Expected result: Response: ok

Command: `curl $SERVICE_URL/health`

STEP 16: Generate demo error requests

- What to do:

Run: `for i in 1 2; do curl -i $SERVICE_URL/error; done`

Solution: Expected result: HTTP 500 demo error responses appear.

Command: `for i in 1 2; do curl -i $SERVICE_URL/error; done`

STEP 17: Open Cloud Logging

- What to do:

Search for Logs Explorer and open it

Solution: Expected result: Logs Explorer opens.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Search and open Logs Explorer in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 18: Filter Cloud Run logs

- What to do:

Use query: `resource.type="cloud_run_revision"`

`resource.labels.service_name="cloudnative-observability-app"`

Solution: Expected result: Logs from cloudfnative-observability-app displayed.

Filter: resource.type="cloud_run_revision"

resource.labels.service_name="cloudfnative-observability-app"

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Filter Cloud Run logs by service name in Logs Explorer

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 19: Filter error logs

- What to do:

Add `severity>=ERROR` to the log filter

Solution: Expected result: Demo error logs displayed.

Filter: resource.type="cloud_run_revision" ... severity>=ERROR

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Filter error logs by severity in Logs Explorer

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 20: Search structured log field

- What to do:

Search for `jsonPayload.event="demo_error"`

Solution: Expected result: Only demo_error event logs displayed.

Filter: `jsonPayload.event="demo_error"`

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Search structured log field demo_error in Logs Explorer

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 21: View logs using command line

- What to do:

Run: `gcloud logging read 'resource.type="cloud_run_revision" AND resource.labels.service_name="cloudnative-observability-app"' --limit=10 --format=json`

Solution: Expected result: Recent log entries in JSON.

Command: `gcloud logging read ... --limit=10 --format=json`

STEP 22: Open Cloud Monitoring

- What to do:

Search for Monitoring and open Cloud Monitoring

Solution: Expected result: Cloud Monitoring opens.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Search and open Cloud Monitoring in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 23: Open Metrics Explorer

- What to do:

In Cloud Monitoring open Metrics Explorer

Solution: Expected result: Metrics Explorer opens.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Open Metrics Explorer in Cloud Monitoring

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 24: View request count metric

- What to do:

Select Cloud Run Revision metric Request count filtered by service name

Solution: Expected result: Request count chart appears.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: View Cloud Run request count metric in Metrics Explorer

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 25: View request latency metric

- What to do:

Select Cloud Run Request latencies metric filtered by service name

Solution: Expected result: Latency chart appears.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: View Cloud Run request latency metric in Metrics Explorer

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 26: View error-related metrics

- What to do:

View request count grouped or filtered by 5xx response code class

Solution: Expected result: Request counts by response code class identified.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: View error-related metrics by response code in Metrics Explorer

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 27: Create or review a dashboard

- What to do:

Create dashboard `cloudnative-practical-9-dashboard` with request, latency, instance, error charts

Solution: Expected result: Dashboard created or concept reviewed.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Create or review custom monitoring dashboard

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 28: Create or review a log-based metric

- What to do:

Create counter metric `cloudnative_demo_error_count` from `demo_error` logs

Solution: Expected result: Log-based metric created or process reviewed.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Create log-based metric from demo_error logs

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 29: Understand log-based alerting

- What to do:

Review option to create alert from log query for `demo_error` events

Solution: Expected result: You can see how a log query becomes an alert.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Review log-based alerting from Logs Explorer

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 30: Understand metric-based alerting

- What to do:

In Cloud Monitoring Alerting review Create Policy for 5xx threshold

Solution: Expected result: You can see the metric-based alerting workflow.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Review metric-based alerting policy in Cloud Monitoring

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 31: Understand uptime checks

- What to do:

In Cloud Monitoring open Uptime checks and review creation for service URL

Solution: Expected result: You can see the purpose of uptime checks.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Review uptime checks configuration in Cloud Monitoring

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 32: Review Cloud Run service metrics page

- What to do:

Open Cloud Run, select `cloudnative-observability-app`, open Metrics tab

Solution: Expected result: Cloud Run metrics visible.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: View Metrics tab on Cloud Run service page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 33: Review Cloud Run logs page

- What to do:

On Cloud Run service page open Logs tab

Solution: Expected result: Cloud Run logs visible.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: View Logs tab on Cloud Run service page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 34: Delete optional alerting resources

- What to do:

Delete test alert policies, uptime checks, log-based metrics, dashboards if created

Solution: Expected result: Optional monitoring resources deleted or reviewed.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Delete optional test alerting and dashboard resources

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 35: Delete Cloud Run service

- What to do:

Run: `gcloud run services delete $SERVICE --region $REGION`

Solution: Expected result: Cloud Run service deleted.

Command: `gcloud run services delete $SERVICE --region $REGION`

STEP 36: Optional cleanup of source deployment images

- What to do:

Open Artifact Registry and delete only practical images if instructed

Solution: Expected result: Practical-related images reviewed or deleted.

Open in Google Cloud Console: <https://console.cloud.google.com/logs/query>

On screen: Open Artifact Registry and delete practical deployment images

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 9` on a machine with your GCP account to capture this step.

STEP 37: Confirm service cleanup

- What to do:

Run: `gcloud run services list --region $REGION`

Solution: Expected result: `cloudnative-observability-app` no longer appears.

Command: `gcloud run services list --region $REGION`

Practical 10: Develop And Deploy Serverless Functions

Objective: In this practical, you will develop, deploy, test, observe, and clean up a serverless HTTP function using Google Cloud Run functions.

STEP 1: Open Google Cloud Console

- What to do:

Open <https://console.cloud.google.com> and sign in

Solution: Expected result: Google Cloud Console opens successfully.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Open and sign in to Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 2: Select the correct project

- What to do:

Use project selector for the practical project

Solution: Expected result: The correct project is selected.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Select the correct project using the project selector

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 3: Open Cloud Shell

- What to do:

Open Cloud Shell from Google Cloud Console

Solution: Expected result: Cloud Shell opens.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Click Cloud Shell icon in Google Cloud Console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 4: Check active account and project

- What to do:

Run: `gcloud auth list; gcloud config get-value project`

Solution: Expected result: Correct account and project displayed.

Commands: `gcloud auth list; gcloud config get-value project`

STEP 5: Set environment variables

- What to do:

Run: `export PROJECT_ID, REGION, FUNCTION_NAME` variables

Solution: Expected result: Variables set.

Commands: `export PROJECT_ID=$(gcloud config get-value project); export REGION=us-central1; export FUNCTION_NAME=cloudnative-http-function`

STEP 6: Enable required APIs

- What to do:

Run: `gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com logging.googleapis.com`

Solution: Expected result: Required APIs enabled.

Commands: `gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com logging.googleapis.com`

STEP 7: Create function folder

- What to do:

Run: `mkdir cloudnative-serverless-function && cd cloudnative-serverless-function`

Solution: Expected result: Function folder created.

Commands: `mkdir cloudnative-serverless-function; cd cloudnative-serverless-function`

STEP 8: Create package.json

- What to do:

Create package.json with Functions Framework dependency and helloHttp target

Solution: Expected result: package.json created.

STEP 9: Create index.js

- What to do:

Create index.js registering helloHttp HTTP function with structured logging

Solution: Expected result: index.js created.

STEP 10: Review the function code

- What to do:

Run: `cat index.js`

Solution: Expected result: Function code displayed.

Command: `cat index.js`

STEP 11: Deploy the HTTP function

- What to do:

Run: `gcloud run deploy $FUNCTION_NAME --source . --function helloHttp --base-image nodejs24 --region $REGION --allow-unauthenticated`

Solution: Expected result: Function built and deployed.

Command: `gcloud run deploy $FUNCTION_NAME --source . --function helloHttp --base-image nodejs24 --region $REGION --allow-unauthenticated`

STEP 12: Save function URL

- What to do:

Run: `export FUNCTION_URL=$(gcloud run services describe $FUNCTION_NAME --region $REGION --format='value(status.url)'); echo $FUNCTION_URL`

Solution: Expected result: Function URL displayed.

Commands: `export FUNCTION_URL=...; echo $FUNCTION_URL`

STEP 13: Test the function

- What to do:

Run: `curl $FUNCTION_URL`

Solution: Expected result: Function returns greeting message.

Command: `curl $FUNCTION_URL`

STEP 14: Test with query parameter

- What to do:

Run: `curl "$FUNCTION_URL?name=Student"`

Solution: Expected result: Hello Student! This response is from a serverless Cloud Run function.

Command: `curl "$FUNCTION_URL?name=Student"`

STEP 15: Test from browser

- What to do:

Open function URL in browser with optional ?name=Cloud query parameter

Solution: Expected result: Browser displays function response.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Open Cloud Run function URL in browser to test HTTP response

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 16: View function logs

- What to do:

Run: `gcloud run services logs read $FUNCTION_NAME --region $REGION --limit=20`

Solution: Expected result: Recent function logs displayed.

Command: `gcloud run services logs read $FUNCTION_NAME --region $REGION --limit=20`

STEP 17: Open Logs Explorer

- What to do:

Search Logs Explorer; filter by `cloud_run_revision` and service name

Solution: Expected result: Logs for the function visible.

Filter: `resource.type="cloud_run_revision"`

`resource.labels.service_name="cloudnative-http-function"`

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Open Logs Explorer and filter function logs

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 18: Filter structured log event

- What to do:

Search for `jsonPayload.event="function_invoked"`

Solution: Expected result: Function invocation logs filtered.

Filter: `jsonPayload.event="function_invoked"`

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Filter function_invoked structured logs in Logs Explorer

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 19: View function in Cloud Run

- What to do:

Open Cloud Run and select `cloudnative-http-function`

Solution: Expected result: Function service details visible.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Open cloudnative-http-function in Cloud Run console

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 20: Review metrics

- What to do:

Open Metrics tab; observe request count, latency, instance count, errors

Solution: Expected result: Metrics visible for the function.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: View Metrics tab on Cloud Run function service page

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 21: Understand authenticated function option

- What to do:

Review `--no-allow-unauthenticated` for production deployments

Solution: Expected result: You can see the difference between public and authenticated functions.

STEP 22: Understand service account option

- What to do:

Review `--service-account SERVICE_ACCOUNT_EMAIL` for least privilege

Solution: Expected result: You can see that functions have runtime identities.

STEP 23: Understand secrets option

- What to do:

Review using Secret Manager instead of hard-coded secrets

Solution: Expected result: You can see the purpose of Secret Manager.

STEP 24: Optional local framework concept

- What to do:

Review `npm install` & `npm start` for local Functions Framework testing

Solution: Expected result: You can see Functions Framework supports local testing.

STEP 25: Optional event-driven function concept

- What to do:

Review Pub/Sub, Cloud Storage, Firestore, Eventarc triggers

Solution: Expected result: You can see difference between HTTP and event-driven functions.

STEP 26: Delete the function

- What to do:

Run: `gcloud run services delete $FUNCTION_NAME --region $REGION`

Solution: Expected result: Function service deleted.

Command: `gcloud run services delete $FUNCTION_NAME --region $REGION`

STEP 27: Optional cleanup of build artifacts

- What to do:

Open Artifact Registry and delete only practical images if instructed

Solution: Expected result: Practical-related images reviewed or deleted.

Open in Google Cloud Console: <https://console.cloud.google.com/run>

On screen: Open Artifact Registry and delete practical function build images

[Screenshot pending] Run `capture_gcp_practical_screenshots.py --login --practical 10` on a machine with your GCP account to capture this step.

STEP 28: Confirm cleanup

- What to do:

Run: `gcloud run services list --region $REGION`

Solution: Expected result: `cloudnative-http-function` no longer appears.

Command: `gcloud run services list --region $REGION`